VGG와 ResNet

3×3을 깊게 쌓기, 그리고 덧셈 하나로 더 깊게 · 책 9.8절 · 19차시

오늘의 질문 두 개

1 VGG

큰 돋보기 하나 대신
작은 3×3을 여러 번 쌓으면
무엇이 좋아질까?

→ 같은 범위를 더 싸게, 더 똑똑하게

2 ResNet

그렇게 좋은데
왜 20층 넘게는 못 쌓았을까?
어떻게 152층까지 쌓았을까?

→ 입력을 그대로 더하는 지름길

예제 파일 (09-cnn)

b16 유명한 모델 크기 · b17 수용 영역 · b18 잔차 블록 · b19 기울기 크기 · b20 21층 학습 비교 · b24 작은 ResNet 실습

1 CNN 구조의 흐름

LeNet → AlexNet → VGG → ResNet

유명한 구조 네 개: 무엇을 새로 가져왔나

모델	해	합성곱층	파라미터	새로 가져온 아이디어
LeNet-5	1998	2	6만	합성곱 → 풀링 → 완전 연결의 기본 틀
AlexNet	2012	5	6,110만	ReLU, 드롭아웃, 데이터 증강, GPU 학습
VGG-16	2014	13	1억 3,840만	**3×3 필터만 깊게 쌓기**
ResNet-18	2015	20	1,170만	**잔차 연결로 수십~수백 층**
ResNet-50	2015	53	2,560만	**+ 병목 블록, 전역 평균 풀링**

흐름 한 줄

더 깊게 쌓을수록 잘 맞힌다 → VGG가 3×3으로 깊게 쌓는 길을 열고 → ResNet이 깊이의 벽을 넘었다

눈여겨볼 숫자

ResNet-50은 VGG-16보다 층이 4배인데 크기는 5분의 1 (b16)

2

VGG: 3×3만 깊게 쌓기

옥스퍼드 VGG 팀 · 2014 · 16층, 19층

VGG-16의 모양: 같은 블록의 반복



규칙은 딱 두 개

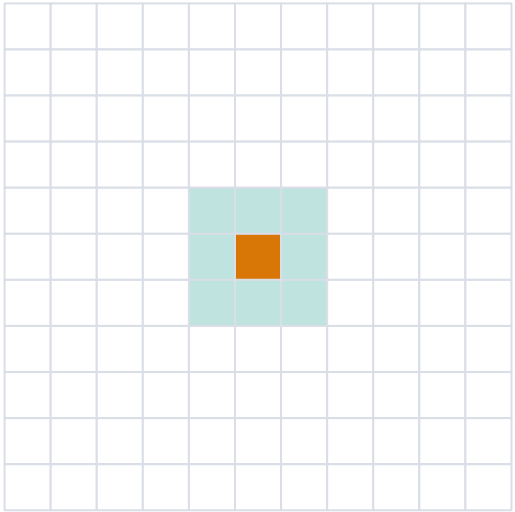
① 합성곱은 전부 3×3, padding=1 (크기 유지)
② 풀링으로 크기를 반으로 줄일 때마다 채널은 두 배 (512에서 멈춤)

9.5절 CNN과 같은 모양

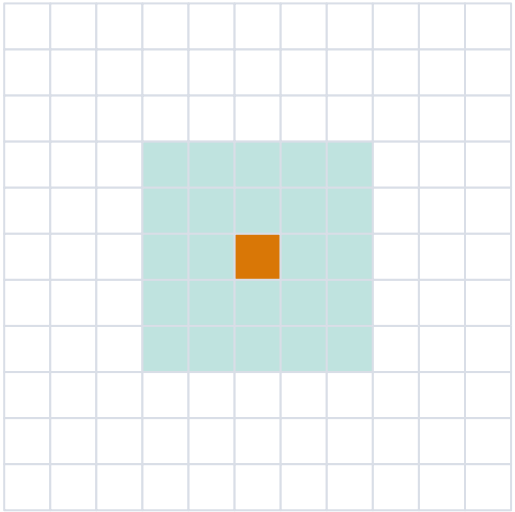
크기는 줄이고 채널은 늘린다
28→14→7, 1→32→64 를
224→7, 3→512 로 길게 한 것

왜 3×3을 쌓나 ①: 쌓으면 보는 범위가 넓어진다

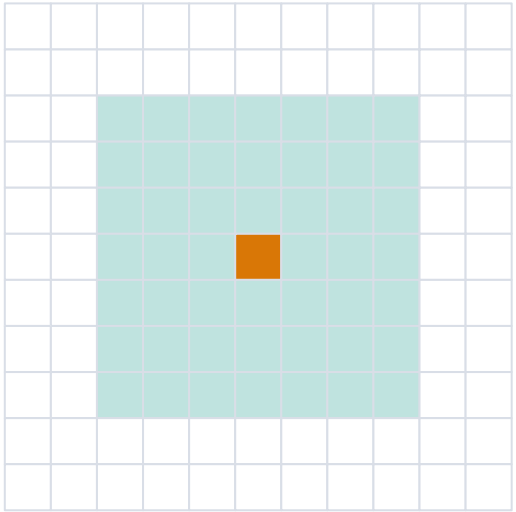
3×3 한 번 → 3×3



3×3 두 번 → 5×5



3×3 세 번 → 7×7



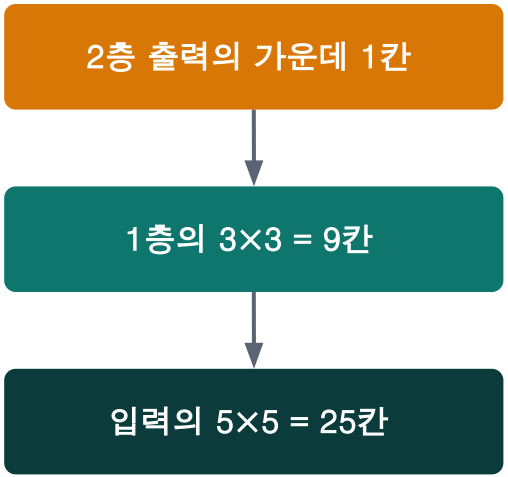
읽는 법

주황 칸 = 출력의 가운데 한 칸 / 초록 칸 = 그 한 칸을 만들 때 쓰인 입력 칸
한 번 쌓을 때마다 위·아래·왼쪽·오른쪽으로 한 칸씩 넓어진다 → n번이면 (2n+1)×(2n+1)

이 범위의 이름

수용 영역 (receptive field)
확인: b17_receptive.py

왜 3×3을 쌓나 ②: 거꾸로 따라가 보면



둘째 3×3이 본 곳

9칸이 각자 입력의 3×3을 봄
→ 겹쳐 보면 양쪽 한 칸씩 더 넓음

```
def 보는_범위(층, 크기=11):
    x = torch.randn(1, 1, 크기, 크기,
                    requires_grad=True)
    출력 = 층(x)
    출력[0, 0, 5, 5].backward()    # 가운데
칸만
    return (x.grad[0, 0] != 0).int()
```

기울기로 찾는 이유

기울기가 0이 아니다 = 그 입력 칸을 바꾸면 출력 칸이 바뀐다 = 계산에 쓰였다
(5장 b05~b12의 backward를 입력 쪽으로 쓴 것)

왜 3×3을 쌓나 ③: 같은 범위를 더 싸게

방법 (채널 64 → 64)	보는 범위	파라미터 계산	파라미터	절약
5×5 한 번	5×5	25 × 64 × 64	102,400	
3×3 두 번	5×5	2 × 9 × 64 × 64	**73,728**	**28% ↓**
7×7 한 번	7×7	49 × 64 × 64	200,704	
3×3 세 번	7×7	3 × 9 × 64 × 64	**110,592**	**45% ↓**

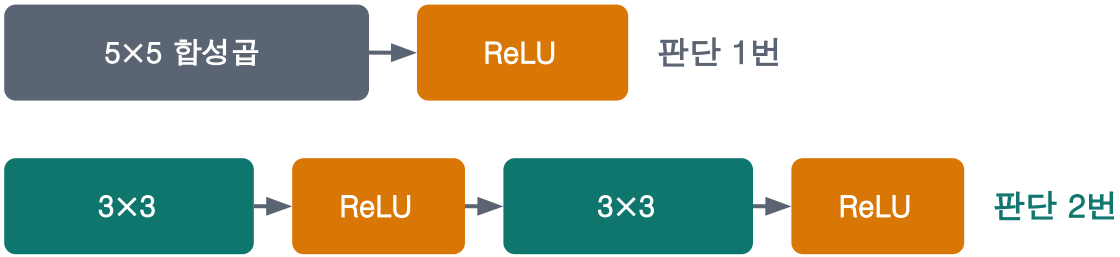
왜 쌓까

큰 필터는 칸 수가 제곱으로 늘어남: 3×3 = 9, 5×5 = 25, 7×7 = 49
작은 필터를 쌓으면 범위는 2칸씩, 비용은 9씩만 늘어남

범위가 넓을수록

아끼는 양이 커진다
5×5 대비 28%, 7×7 대비 45%

왜 3×3을 쌓나 ④: 판단(ReLU)이 더 많아진다



판단이 여러 번이면 단계적으로 조합할 수 있다

1층: 짧은 선, 밝기 경계 (19차시 b21: 첫 층 필터 대부분이 경계 탐지기)
2층: 선이 만나는 모서리, 곡선
3층 이상: 모서리가 모인 모양 → 부위 (b22: 깊은 층은 발등·밑창 같은 부위 지도)

정리: 3×3 쌓기 = 같은 범위 + 더 적은 파라미터 + 더 많은 판단 → 요즘 CNN은 거의 다 3×3

VGG의 한계 두 가지

① 크기: 파라미터의 89%가 끝의 완전 연결층

층	파라미터
합성곱 13개 전부	약 1,470만
Linear(25,088 → 4,096)	**약 1억 280만**
Linear(4,096 → 4,096)	약 1,680만
Linear(4,096 → 1,000)	약 410만

9.5절 CNN의 Flatten + 큰 Linear와 같은 문제

② 깊이: 19층 근처가 끝

깊을수록 좋다면 50층, 100층은?

그냥 더 쌓으면 오히려 나빠진다
(시험뿐 아니라 학습 데이터에서도)

→ 과적합이 아니라 학습 자체가 안 되는 문제

두 문제 모두 ResNet이 풀었다: ①은 전역 평균 풀링, ②는 잔차 연결

확인 문제: VGG

1. 3×3을 4번 쌓으면 출력 한 칸이 보는 입력 범위는?

   (b17의 `for 깊이 in (1, 2, 3)`에 `4` 추가)

2. 채널 32 → 32에서 5×5 한 번과 3×3 두 번의 파라미터는? (bias=False)

3. VGG-16이 1억 3,840만 개나 되는 가장 큰 이유는?

확인 문제 정답: VGG

1

9×9
(2n + 1) = 2×4 + 1 = 9

2

5×5: 25 × 32 × 32 = 25,600
3×3 두 번: 2 × 9 × 32 × 32 = 18,432
채널이 달라도 28% 절약은 같다

3

끝의 완전 연결층 (특히 Linear(25,088 → 4,096) 하나가 1억 개 이상)
3×3 쌓기 때문이 아니다

3 깊게 쌓으면 생기는 문제

더 깊은데 더 못 배운다 · 기울기 소실

깊게 쌓으면 생기는 문제

더 깊은 모델이 학습 데이터조차 못 맞힌다

ResNet 논문(2015)이 보여 준 현상

CIFAR-10에서 평범한 CNN 20층 vs 56층
→ 56층이 시험 오차뿐 아니라 학습 오차도 더 높음

과적합이라면 학습 오차는 더 낮아야 한다
→ 과적합이 아니라 깊은 모델을 학습시키는 것 자체가 어렵다는 뜻

이상한 점

56층은 20층을 흉내 낼 수 있다
(앞 20층은 그대로, 나머지 36층은
입력을 그대로 내보내면 됨)

그런데 그 "그대로 내보내기"를
학습으로 찾지 못한다

"그대로 내보내기"를 배우기 어렵다면 → 아예 구조에 넣어 주자 = ResNet의 출발점

직접 확인: b20_plain_vs_res.py (합성곱 21층 학습 비교, 4부에서)

원인: 앞쪽 층까지 기울기가 거의 오지 않는다

합성곱층 수	평범한 CNN	잔차 CNN
5	4.48e-04	3.54e-02
21	**3.94e-10**	3.62e-02
51	**9.17e-23**	**3.58e-02**

첫 층 기울기 크기 (b19_grad_flow.py)

8장 디버깅 4단계(층별 기울기 크기 보기)에서 본 "소실"이 바로 이것

왜 사라지나 (연쇄 법칙, 5장)

기울기는 뒤에서부터 층마다 곱해지며 내려온다
1보다 작은 값을 수십 번 곱하면
$0.5^{20} \approx 0.000001$ → 앞쪽 층은 거의 안 바뀐다

읽는 법

9.17e−23 = 0.0000…(0이 22개)917
51층 평범한 CNN의 첫 층은 사실상 학습이 멈춤
잔차 CNN은 깊이와 상관없이 비슷한 크기

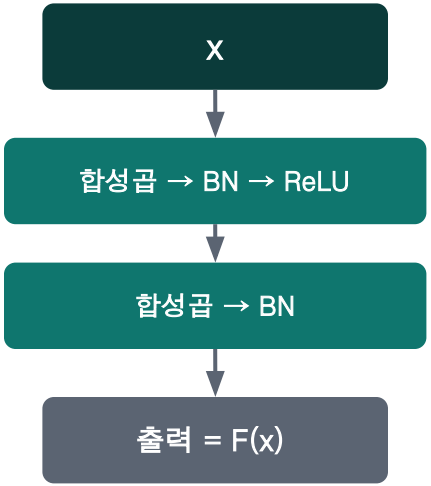
4

ResNet: 고칠 양만 배우기

마이크로소프트 연구소 · 2015 · 152층 · 이미지넷 우승

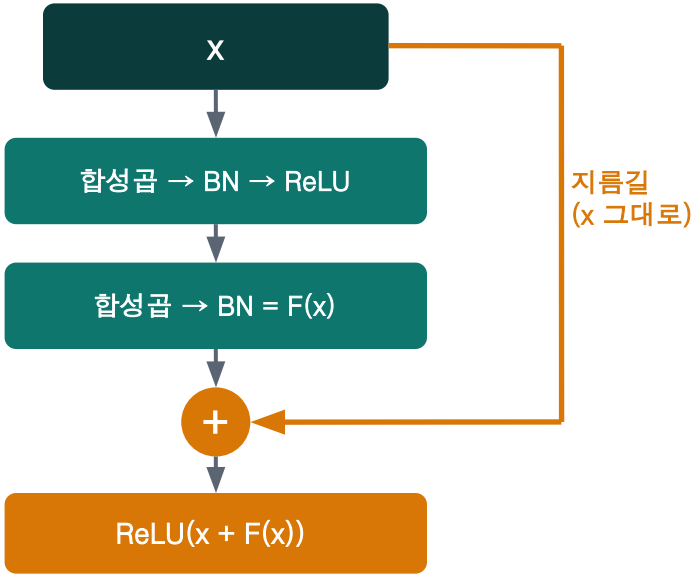
아이디어: 새로 그리지 말고 고칠 부분만 덧칠하기

평범한 블록



출력 전체를 새로 만들어야 함

잔차 블록



덧칠 비유

F(x)는 "고칠 양"만 배우면 된다
고칠 게 없으면 F = 0 → 출력 = x

"잔차(residual)" = 남은 차이 = 정답 − 입력 = 고칠 양

왜 깊게 쌓아도 되나: 기울기 고속도로

출력 = x + F(x) → x로 미분하면 1 + F′(x)

"1 +"의 의미

F′가 아주 작아져도 1은 남는다
→ 기울기가 지름길을 타고 앞쪽 층까지 그대로 내려간다
→ 51층에서도 첫 층 기울기 3.58e-02 (b19)

"아무것도 안 하기"가 쉬워짐

F의 가중치를 0으로만 보내면 출력 = x
→ 필요 없는 블록은 그냥 통과
→ 깊게 쌓아도 얕은 모델보다 나빠지기 어렵다

```
nn.init.zeros_(같은크기.정규화2.weight)    # 고칠 양을 0으로
같은크기.eval()
print(torch.allclose(같은크기(x), F.relu(x)))    # True: 그대로 통과
```

잔차 블록 코드 (b18_resblock.py)

```
class 잔차블록(nn.Module):
    def __init__(self, 입력, 출력, 보폭=1):
        super().__init__()
        self.합성곱1 = nn.Conv2d(입력, 출력, 3, stride=보폭,
                                 padding=1, bias=False)
        self.정규화1 = nn.BatchNorm2d(출력)
        self.합성곱2 = nn.Conv2d(출력, 출력, 3, padding=1, bias=False)
        self.정규화2 = nn.BatchNorm2d(출력)
        if 보폭 != 1 or 입력 != 출력:           # 모양이 바뀌면
            self.지름길 = nn.Sequential(
                nn.Conv2d(입력, 출력, 1, stride=보폭, bias=False),
                nn.BatchNorm2d(출력))
        else:
            self.지름길 = nn.Identity()         # 그대로 통과

    def forward(self, x):
        h = F.relu(self.정규화1(self.합성곱1(x)))
        고칠양 = self.정규화2(self.합성곱2(h))
        return F.relu(고칠양 + self.지름길(x))    # 핵심은 이 덧셈
```

bias=False

바로 뒤 BN이 평균을 빼며 편향을 지움 (9.7, b14)

더하려면 모양이 같아야

보폭 2 → 크기 절반
채널 32 → 64 → 채널 다름
이럴 때 지름길도 1×1 합성곱으로 맞춤

실행 결과

(8,32,14,14)→(8,32,14,14) Identity
(8,32,14,14)→(8,64,7,7) Sequential

실제로 학습해 보면: 합성곱 21층, 5에폭 (b20)

설정	손실	정확도	해석
평범한	**2.302**	**10.1%**	손실 = −ln(1/10): 찍기 수준, 학습이 안 됨
평범한 + BN	0.778	58.7%	BN만으로도 살아남
잔차	1.153	49.5%	덧셈 하나로 살아남
잔차 + BN	**0.597**	**69.2%**	**가장 좋음 = 실제 ResNet의 조합**

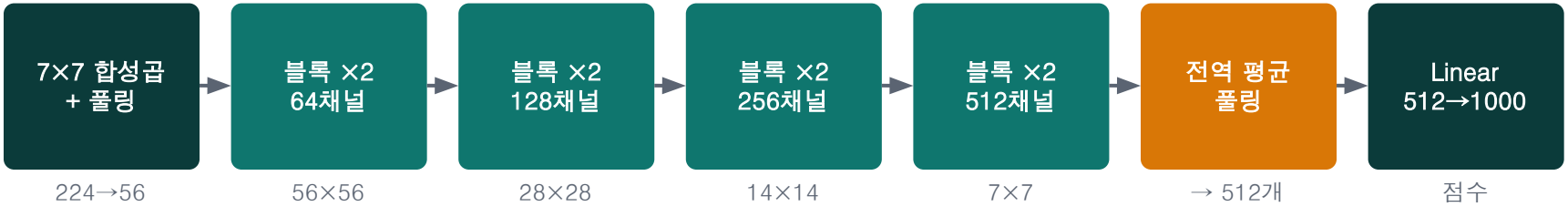
2.302가 뜻하는 것

8장 디버깅 1단계: 처음 손실은 ln(10) = 2.3026
5에폭 뒤에도 그대로 → 처음 상태에서 한 발짝도 못 움직임

조건

학습 5,000장, 14×14로 줄인 사진
SGD + momentum, 씨앗 고정

ResNet 전체 모양



ResNet-18 (블록 안 합성곱 2개 × 8블록 + 첫 층 1 + 지름길 1×1 3개 = 합성곱 20개)

VGG에서 가져온 것

3×3 쌓기
크기 절반 → 채널 두 배

VGG와 다른 것

풀링 대신 보폭 2 합성곱으로 줄임
큰 Linear 대신 전역 평균 풀링
→ 1,170만 개 (VGG-16의 1/12)

ResNet-50 이상

병목 블록: 1×1로 채널 줄이고
→ 3×3 → 1×1로 다시 늘림
싸게 더 깊게 (53층, 152층)

20차시 연결: 작은 ResNet으로 옷 분류

```
def 작은_ResNet():
    return nn.Sequential(
        nn.Conv2d(1, 32, 3, padding=1, bias=False), nn.BatchNorm2d(32), nn.ReLU(),
        잔차블록(32, 32), 잔차블록(32, 64, 보폭=2), 잔차블록(64, 128, 보폭=2),
        nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Dropout(0.3), nn.Linear(128, 10),
    )
```

모델	파라미터	시험 정확도	4칸 밀기
6장 MLP	235,146	88.58%	25.2%
9.5절 CNN (Flatten + 큰 Linear)	421,642	91.01%	36.0%
작은 ResNet	**308,074**	**93.04%**	**92.0%**

파라미터는 더 적은데 더 잘 맞히고, 밀어도 버틴다 (전역 평균 풀링 + 증강)

확인 문제: ResNet

1. 잔차블록(32, 32, 보폭=2)의 출력 모양과 지름길 종류는? (입력 (8, 32, 14, 14))

2. forward를 return F.relu(고칠양)으로 바꾸면 "고칠 양이 0이면 그대로 통과"는?

3. 평범한 21층 CNN의 손실이 5에폭 내내 2.30인 이유는?

4. x + F(x)를 x로 미분하면? 이것이 왜 중요한가?

확인 문제 정답: ResNet

1 (8, 32, 7, 7), Sequential
채널이 같아도 크기가 절반이 되므로 지름길도 보폭 2인 1×1 합성곱으로 맞춰야 한다

2 False. 출력이 전부 0
지름길이 없으면 입력을 넘겨줄 길이 없다

3 기울기 소실로 앞쪽 층이 거의 안 바뀜
2.30 = −ln(1/10) = 10개 중 찍기 수준의 손실

4 1 + F′(x)
F′가 작아져도 1이 남아 기울기가 앞쪽까지 그대로 전달된다

정리

01	VGG	3×3만 쌓는다. 두 번이면 5×5, 세 번이면 7×7을 본다
02	3×3 쌓기의 이득	같은 범위를 28~45% 적은 파라미터로, ReLU 판단은 더 많이
03	VGG의 한계	끝의 큰 Linear가 파라미터의 89% · 19층 넘게는 학습이 안 됨
04	깊이의 벽	기울기 소실: 21층 평범한 CNN은 손실 2.30에서 안 움직임
05	ResNet	출력 = x + F(x). 고칠 양만 배우고, 지름길로 기울기가 흐른다
06	요즘 CNN의 기본	3×3 + BN + 잔차 연결 + 전역 평균 풀링 (20차시 작은 ResNet)